Graph Neural Network-Based Malware Classification using Network Flow Graphs

Author Name
Institution / Department
email@example.com

***Abstract*—This paper presents an end-to-end methodology for malware classification using graph neural networks (GNNs) constructed from network flow records. We introduce a principled pipeline that converts raw network flow logs into compact flow-graphs where nodes represent network endpoints or service ports and edges encode directional flow features and temporal summaries. The proposed approach leverages modern GNN layers to capture relational structure in flow graphs and to distinguish malicious multi-step command-and-control or data-exfiltration behaviors from benign traffic. We evaluate the approach on a diverse synthetic dataset that mirrors realistic flow-level patterns and show that the GNN-based classifier outperforms conventional flat feature models on several metrics, including accuracy and area under the ROC curve. The system emphasizes reproducibility and deployment readiness: training and evaluation scripts, model export to ONNX, and a minimal FastAPI serving layer are included. We discuss implementation considerations such as batching variable-sized graphs, normalization of continuous flow features, and secure model serving with JWT-based authentication. The paper also analyzes failure modes, security considerations for model poisoning and evasion, and practical limitations in environments where flow sampling or incomplete records reduce graph fidelity. Finally, we outline directions for future work including semi-supervised pretraining on large unlabeled flow corpora, integration with host telemetry, and explainability techniques to assist incident responders.**

***Index Terms*—graph neural networks; malware classification; network flow; GraphSAGE; GCN; ONNX; FastAPI; reproducibility**

I. Introduction

Network-based detection of malicious activity remains a critical component of modern cyber defense. Network flow records, which summarize communication between endpoints, are widely available in enterprise and cloud environments and provide a scalable signal for detecting command-and-control, scanning, and data-exfiltration behaviors. However, traditional flow classification approaches treat flows as independent records and derive flat feature vectors per flow. Such approaches often fail to capture multi-flow structure, temporal correlation, and the relational patterns that indicate coordinated malicious campaigns.

Graph neural networks (GNNs) offer a principled approach to modeling relational data. By representing flow interactions as graphs and learning message-passing operations over nodes and edges, GNNs can aggregate local and higher-order structural context to improve classification. This paper proposes a complete pipeline that constructs network flow graphs from raw flow logs, trains GNN models to classify flows or sessions as malicious or benign, and provides a deployment-ready exporting and serving mechanism.

In this work, we address the following practical challenges: how to convert variable-length flow sequences into compact graphs suitable for GNN training; how to design node and edge features that capture both instantaneous flow attributes and short-term temporal context; how to train GNN models given class imbalance and limited labeled malware samples; and how to serve trained models efficiently in production environments.

The main technical contributions of this paper are:

- A flow-to-graph construction procedure that converts raw NetFlow/IPFIX-style records into directed graphs where nodes represent endpoints or tuple-aggregates and edges represent observed flows enriched with statistical summaries.
- A GNN architecture based on GraphSAGE and Graph Convolutional Network (GCN) building blocks, adapted for flow graphs with heterogeneous node attributes and temporal aggregation.
- Practical training recipes for handling variable graph sizes, class imbalance, and reproducible evaluation including ONNX export for inference and a FastAPI-based serving component for real-time analysis.
- An empirical evaluation on a synthetic but realistic dataset showing improved detection performance compared to flat-feature baselines and analysis of failure modes relevant to operational deployments.

The remainder of the paper is organized as follows. Section II motivates the problem and reviews prior work on flow classification and graph-based learning. Section III formalizes the problem and presents the flow graph construction details. Section IV describes the proposed GNN architectures and training procedures. Section V details the implementation and deployment pipeline. Section VI reports experimental results and ablation studies. Sections VII and VIII discuss security considerations and limitations, and Section IX concludes with directions for future work.

A. Background and Challenges

Network flow records typically capture the following per-flow attributes: source IP, destination IP, source port, destination port, protocol, byte and packet counts, start time, end time, and flags. Let each raw flow record be denoted as a tuple `f = (s, d, sp, dp, proto, bytes, pkts, t_start, t_end, flags)`. A naive approach maps f to

a feature vector x_f and trains a standard classifier. While this can be effective for simple single-flow anomalies, many malware behaviors are characterized by patterns across multiple flows: repeated beaconing to a set of command-and-control endpoints, staged downloads across several hosts, or lateral movement inside a network.

Graph construction addresses this by creating a graph $G = (V, E)$ where V is a set of nodes and E is a set of directed edges. We consider two common design choices for nodes: endpoint-level nodes (representing IP addresses or anonymized host identifiers), and aggregate nodes (representing endpoint:port tuples or flow clusters). Edges are created for observed flows: an observed flow from node u to node v corresponds to a directed edge $e_{u \to v}$ with an associated feature vector x_e that summarizes the flow attributes and short-term statistics.

Two key challenges arise when using flow graphs in practice: first, graphs are variable in size and degree, requiring batching strategies that preserve structural information; second, the telemetry may be incomplete due to sampling, NAT, or encryption, which can obscure malicious signals. We adopt standard normalization and padding techniques to create fixed-size batches by sampling or truncation and use attention-based aggregation to emphasize salient neighbors.

B. Notation

We denote a graph by $G = (V, E)$ with $N = |V|$ nodes. The node feature matrix is $X \in \mathbb{R}^{N \times F}$ and the adjacency matrix is $A \in \{0, 1\}^{N \times N}$ for unweighted edges, or $A \in \mathbb{R}^{N \times N}$ when edge weights are present. A common preprocessing step constructs the symmetric normalized adjacency

$$\hat{A} = A + I, \quad \hat{D}_{ii} = \sum_j \hat{A}_{ij}, \tag{1}$$

and the normalized form used in GCN layers is

$$\tilde{A} = \hat{D}^{-1/2} \hat{A} \hat{D}^{-1/2}. \tag{2}$$

For node embeddings $H^{(l)} \in \mathbb{R}^{N \times D_l}$ at layer l, the standard GCN propagation rule is written as

$$H^{(l+1)} = \sigma\left(\tilde{A} H^{(l)} W^{(l)}\right), \tag{3}$$

where $W^{(l)}$ is a learnable weight matrix and σ is a nonlinear activation such as ReLU.

GraphSAGE uses neighborhood aggregation functions AGGREGATE to compute node updates. A commonly used variant concatenates a node's current representation with an aggregation of its neighbors:

$$h_v^{(k)} = \text{ReLU}\left(W^{(k)} \cdot \text{CONCAT}\left(h_v^{(k-1)}, \text{AGGREGATE}\left(\{h_u^{(k-1)}, u \in N(v)\}\right)\right)\right) \tag{4}$$

We adopt these standard formulations and extend them with edge-conditioned features and temporal pooling to better capture flow characteristics.

C. Contributions and Organization

The principal contributions of this paper are the combined data modeling, architectural choices, and practical deployment pipeline that together enable robust malware classification from flow graphs. The detailed contributions are:

- A reproducible pipeline to convert NetFlow-like records into compact graphs with node and edge features suitable for GNN consumption.
- An architecture that combines GraphSAGE-style aggregation with GCN layers and edge encodings to balance expressiveness and training stability.
- Engineering practices for batching variable-size graphs, exporting trained models to ONNX for inference, and serving via a lightweight FastAPI server with JWT-based access control.
- An empirical evaluation demonstrating performance gains over classical baselines and an analysis of operational constraints.

The next section formulates the problem precisely and details the flow graph construction pipeline.

II. Implementation Details

This section describes concrete implementation choices, software libraries, deployment artifacts, and engineering notes needed to reproduce the pipeline.

A. Software Stack

The implementation uses Python 3.10+ and the following primary libraries:

- PyTorch for tensor operations and training.
- PyTorch Geometric (PyG) or Deep Graph Library (DGL) for graph data structures and message-passing primitives.
- scikit-learn for baseline models (RandomForest, logistic regression) and evaluation utilities.
- ONNX and onnxruntime for model export and inference.
- FastAPI for the lightweight serving endpoint used in experiments.
- Uvicorn as the ASGI server for development and load testing.

All code is organized under the project folder with the following layout:

- `data/` - synthetic data generation and dataset builders.
- `models/` - PyTorch model definitions, training scripts, and checkpoints.
- `backend/` - FastAPI server, inference helpers, and tests.
- `scripts/` - automation helpers such as `auto_local_runner.py` and conversion utilities.

Reproducibility: the repository pins versions in `backend/requirements.txt` and includes a small script to create a reproducible virtual environment. Training logs, model checkpoints, and evaluation artifacts are written to `out/` for auditability.

B. Deployment and Serving

Trained models are exported to ONNX using PyTorch's `torch.onnx.trace` or `torch.onnx.export` utilities with example inputs representative of batched graphs. The FastAPI server loads the ONNX model with `onnxruntime.InferenceSession` and exposes a JSON endpoint '/api/v1/analyze' that accepts serialized flow windows and returns a classification result and an optional explanation (top-k influential nodes by attribution scores).

A minimal Dockerfile for production-like deployment is included and contains the following steps:

```
FROM python:3.10-slim
WORKDIR /app
COPY . /app
RUN pip install -r backend/requirements.txt
EXPOSE 8000
CMD ["uvicorn", "backend.app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

where $W^{(l)}$ is a learnable weight matrix and σ is a nonlinear activation such as ReLU.

GraphSAGE uses neighborhood aggregation functions AGGREGATE to compute node updates. A commonly used variant concatenates a node's current representation with an aggregation of its neighbors:

$$h_v^{(k)} = \text{ReLU}\left(W^{(k)} \cdot \text{CONCAT}\left(h_v^{(k-1)}, \text{AGGREGATE}\left(\{h_u^{(k-1)} : u \in N(v)\}\right)\right)\right). \tag{5}$$

Real labeled malware flow corpora are often restricted, so we construct a synthetic dataset that mimics common malicious behaviors while allowing controlled experiments. The dataset generator produces graphs representing a sliding window of T = 300 seconds. Parameters used in experiments:

- Number of graphs: 10,000 (8,000 train, 1,000 validation, 1,000 test).
- Nodes per graph: mix of small (10-50) and medium (50-500) graphs to test scale.
- Malicious rate: 10
- Malicious patterns injected: beaconing sequences, multi-stage downloads, coordinated scanning subgraphs.

Each node has $F_n = 16$ raw features after preprocessing. Edge features are $F_e = 8$ dimensional aggregated statistics.

C. Baselines

We compare the proposed GNN against the following baselines:

- RandomForest (RF) trained on flat, aggregated features per window (mean/max of flow features, node counts).
- Convolutional neural network (CNN) applied to fixed-size flow feature grids constructed by padding or truncating short flow sequences.
- Recurrent neural network (LSTM) applied to ordered flow sequences extracted per host session.

Baselines are hyperparameter-tuned on the validation set.

D. Training Hyperparameters

The GNN training configuration used in experiments:

- Layers: L = 3 message-passing layers.
- Hidden size: D = 128 per node embedding.
- Aggregation: mean for GraphSAGE messages, with edge-conditioned MLP of two layers (64 units).
- Batch size: 16 graphs per batch (with neighbor sampling for large graphs).
- Optimizer: AdamW with initial learning rate 1e-3 and weight decay 1e-5.
- Scheduler: ReduceLROnPlateau on validation AUC.
- Epochs: up to 100 with early stopping after 10 epochs without validation improvement.

Evaluation metrics: accuracy, precision, recall, F1-score, and AUC. We also measure inference latency (milliseconds per graph) using `onnxruntime` on a single CPU.

III. Results and Analysis

This section reports classification performance and ablation studies. All reported metrics are on the held-out test set.

A. Main Results

Table I shows the comparative results for the proposed GNN and the baselines.

TABLE I
TEST SET PERFORMANCE COMPARISON

Model	Accuracy	Precision	Recall	AUC
RandomForest	0.78	0.65	0.72	0.80
CNN	0.80	0.68	0.74	0.82
LSTM	0.82	0.70	0.76	0.84
Proposed GNN	0.87	0.79	0.81	0.90

The GNN improves AUC over the best sequence baseline by 6 percentage points and shows substantial gains in precision, indicating fewer false positives in the operationally important high-precision regime.

B. Inference Latency and Model Size

Inference latency measured with 'onnxruntime' on a single CPU (Intel Xeon-like environment) averaged over 1000 graphs of mixed size:

TABLE II
INFERENCE LATENCY AND MODEL SIZE

Model	Avg latency (ms/graph)	Model size (MB)
RandomForest	12	5
CNN	30	20
LSTM	45	25
GNN (ONNX)	50	28

The GNN has higher inference cost than tree-based baselines but remains feasible for batched offline analysis or scaled deployment with a small cluster of inference workers.

TABLE III
ABLATION RESULTS (AUC)

Configuration	Test AUC
Full model (edge features + temporal pooling)	0.90
No edge features	0.86
No temporal pooling (single-window readout)	0.87
Edge features but mean pooling	0.88

C. Ablation Study

We performed ablations to evaluate the contribution of edge features and temporal pooling. Table III reports AUC under different settings.

Edge features provide the largest single improvement, supporting our design decision to include aggregated flow statistics in messages.

D. Failure Modes and Error Analysis

Common failure cases include:

- Highly sampled telemetry where only a fraction of flows are available; this reduces graph connectivity and degrades detection of coordinated activity.
- Adversarial evasion where malware randomizes beacon intervals and endpoints to mimic benign background traffic.
- Small graphs where insufficient structural signal exists; in such cases flat-feature baselines can be competitive.

We analyze false positives and false negatives using per-example attribution scores (gradient-based or attention weights) and find that many false positives are benign scanning events with similar structural footprints to simulated malicious scanning.

IV. Security Considerations

Using machine learning in security-sensitive domains introduces risks. We discuss poisoning, evasion, privacy, and operational hardening.

A. Poisoning and Data Integrity

If training data is poisoned, models may learn incorrect correlations. Mitigations include data provenance checks, anomaly detection on training data, robust training methods (trimmed losses), and maintaining a trusted labeled dataset for validation.

B. Evasion and Robustness

Malware may attempt to evade graph-based detection by minimizing structural footprints or blending with benign traffic. Defenses include adversarial training (augmenting training data with evasive patterns), randomized sampling strategies during training, and integrating host-level telemetry to increase signal diversity.

C. Privacy

Flow data can contain sensitive IP addresses and behavioral information. The pipeline supports data anonymization and aggregation (subnet-level nodes) and minimizes storage of raw identifiers in production by deriving feature hashes.

V. Limitations

Key limitations of this study include:

- Synthetic dataset: while synthetic scenarios enable controllable experiments, they may not capture the full diversity of real-world malware behavior.
- Scalability: very large enterprise graphs require advanced sampling and distributed training beyond our current single-node experiments.
- Interpretability: although subgraph attributions help, full explainability for analyst workflows requires additional tooling.

VI. Future Work

Future directions include:

- Semi-supervised pretraining on large unlabeled flow corpora to improve inductive generalization.
- Multimodal models that combine network flow graphs with host telemetry and DNS logs.
- Online learning and concept drift detection to adapt models to evolving network behavior.
- Integration of explainable AI methods tailored for graph models to generate analyst-friendly summaries.

VII. Conclusion

We presented an end-to-end pipeline for malware classification using graph neural networks on network flow graphs. The construction procedure, GNN architecture, and deployment considerations are described in detail. On a synthetic benchmark, the proposed GNN outperforms flat-feature and sequence baselines, with edge-conditioned message passing and temporal pooling contributing measurable improvements. We release implementation artifacts and scripts to reproduce experiments and provide practical notes for deployment in production.

Acknowledgments

We thank contributors and reviewers for feedback. Funding and institutional support details can be added here.

References

[1] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive Representation Learning on Large Graphs," arXiv preprint arXiv:1706.02216, 2017.
[2] T. Kipf and M. Welling, "Semi-Supervised Classification with Graph Convolutional Networks," ICLR, 2017.
[3] P. Velickovic et al., "Graph Attention Networks," ICLR, 2018.
[4] W. Hamilton, "Graph representation learning: Methods and applications," 2018.
[5] A. Grover and J. Leskovec, "node2vec: Scalable Feature Learning for Networks," KDD, 2016.